MODULE 2 · LECTURE 12-b

CS 6530 Applied Cryptography

RSA

Module 3 – Public-Key Cryptography

RSA Key Generation

RSA — Rivest, Shamir and Adleman (1977)
Introduced at MIT; published 1978. RSA takes its name from their surnames.

RSA brings together the mathematical ideas we have just developed to construct a public/private key pair.

1 — Choose two distinct large primes → p, q

↓

2 — Construct the modulus → $n = pq$

↓

3 — Compute Euler's totient → $\phi(n)=(p-1)(q-1)$

↓

4 — Choose e, with $1 < e < \phi(n)$ → $\gcd(e,\phi(n))=1$

↓

5 — Compute the private exponent d → $d=e^{-1} \bmod \phi(n)$

equivalently, $ed \equiv 1 \pmod{\varphi(n)}$

PUBLIC KEY

$PK = (n, e)$

PRIVATE KEY

$SK = (n, d)$

Key idea: e and d are modular inverses with respect to φ(n), creating the mathematical relationship between the RSA public and private keys.

Worked Example 1 — Generate the RSA Keys

Start with two small primes so that every RSA step can be computed by hand.

Step 1 — Choose p and q

$p = 11$ $q = 13$

Step 2 — Compute the modulus

$$n = pq = 11 \times 13$$

$n = 143$

Step 3 — Compute the totient

$$\phi(n)=(p-1)(q-1)=(11-1)(13-1)=10\times12$$

$\phi(n) = 120$

Step 4 — Choose e

$e = 7$. Check: $\gcd(7, 120) = 1 \rightarrow e$ has an inverse mod 120.

$e = 7$

Step 5 — Find d

$$7 \times 103 = 721 = 6(120) + 1 \rightarrow 7d \equiv 1 \pmod{120}$$

$d = 103$

PUBLIC

$PK = (143, 7)$

PRIVATE

$SK = (143, 103)$

Key idea: RSA key generation converts two primes (p, q) into a public exponent e and its corresponding private exponent d.

Worked Example 1 — Encrypt Message M

From the previous slide: **PK = (143, 7)**

Choose the message representative:

$$M = 9 \quad (0 \le M < n)$$

RSA Encryption

$$C = M^e \bmod n$$

Therefore: $C = 9^7 \bmod 143$

$9^2 \equiv 81, \quad 9^4 \equiv 81^2 \equiv 6561 \equiv 126 \pmod{143}$

$7 = 4 + 2 + 1 \rightarrow 9^7 = 9^4 \cdot 9^2 \cdot 9$

$C \equiv 126 \cdot 81 \cdot 9 \equiv 53 \cdot 9 \equiv 48 \pmod{143}$

$$C = 48$$



This is textbook RSA using a small integer message representative for understanding the mathematics. Real RSA requires appropriate encoding/padding, addressed when we study secure RSA use.

Key idea: RSA encryption raises M to the public exponent e and reduces the result modulo n.

Worked Example 1 — Decrypt and Recover M

We have: $C = 48$ and the private key: $SK = (143, 103)$

$$M' = C^d \bmod n$$

Therefore: $M' = 48^{103} \bmod 143$

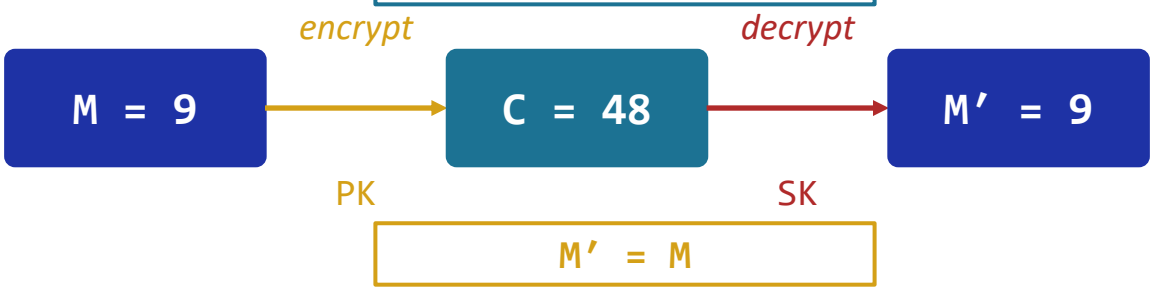
Decompose the exponent: $103 = 64 + 32 + 4 + 2 + 1$

Power	Value mod 143
48^1	48
48^2	16
48^4	113
48^8	42
48^{16}	48
48^{32}	16
48^{64}	113

$$48^{103} = 48^{64} \cdot 48^{32} \cdot 48^4 \cdot 48^2 \cdot 48$$

$M' \equiv 113 \cdot 16 \cdot 113 \cdot 16 \cdot 48 \pmod{143}$ — progressively reducing modulo 143 gives:

$$M' = 9$$



Key idea: The private exponent reverses the RSA public-key operation and recovers the original message.

Why Does RSA Recover the Original Message?

e and d were chosen so that: $d = e^{-1} \bmod \phi(n)$

1. KEY RELATIONSHIP

$$ed = 1 + k\phi(n)$$

Encrypt: $C \equiv M^e \pmod n$ Decrypt: $M' \equiv C^d \equiv M^{ed} \pmod n$

Substitute $ed = 1 + k\phi(n)$: $M^{ed} = M \cdot (M^{\phi(n)})^k$

2. EULER'S THEOREM

For gcd(M, n) = 1:

$$M^{\phi(n)} \equiv 1 \pmod n$$

Therefore: $M' \equiv M \cdot (1)^k \equiv M \pmod n$

3. RSA CORRECTNESS

$$M^{ed} \equiv M \pmod n$$

For our example: $e = 7$, $d = 103$, $\phi(143) = 120$

$7 \times 103 = 721 = 1 + 6(120)$

$$ed = 1 + 6\phi(143)$$

$$D_{SK}(E_{PK}(M)) = M$$

This covers gcd(M, n) = 1; the general case uses the Chinese Remainder Theorem (CRT).

Key idea: RSA works because e and d are chosen so that their product has exactly the modular structure needed to recover M.

Worked Example 2 — Can You Complete RSA?

Use the complete RSA process with a second set of small values.

Given: p=5, q=11, e=3, M=7
Find: n, ϕ(n), d, C, M′

1 — $n = pq = 5 \times 11$

$n = 55$

2 — $\phi(n)=(5-1)(11-1)=4\times10$

$\phi(n) = 40$

3 — $3 \times 27 = 81 = 2(40) + 1 \rightarrow 3d \equiv 1 \pmod{40}$

$d = 27$

PK = (55, 3) SK = (55, 27)

4 — $C = 7^3 \bmod 55 = 343 \bmod 55$

$C = 13$

5 — $13^2 \equiv 4,\ 13^4 \equiv 16,\ 13^8 \equiv 36,\ 13^{16} \equiv 31 \pmod{55};\ 27=16+8+2+1$

$13^{27} \equiv 31 \cdot 36 \cdot 4 \cdot 13 \pmod{55}$

$M' = 7$

M = 7 → Encrypt (PK) → C = 13 → Decrypt (SK) → M′ = 7

CHALLENGE — SPOT THE PROBLEM

Encrypt M = 7 again, same PK: $C = 7^3 \bmod 55 = 13$

7 → 13 7 → 13

Same message + same public key → same ciphertext

Would replacing this toy key with RSA-2048 change this property? **No.**

Key idea: Textbook RSA is deterministic. Increasing the RSA key size does not remove this property.

RSA at Real-World Scale — Key Size and Performance

Our examples use tiny numbers for hand computation. Practical RSA operates on much larger integers.

RSA-k refers to the bit length of the modulus n = pq

It does not mean that p and q are each k bits.

RSA Designation	Modulus n	Approx. Digits of n	Approx. Prime Size
RSA-1024	1024 bits	308 digits	~512 bits
RSA-2048	2048 bits	617 digits	~1024 bits
RSA-3072	3072 bits	925 digits	~1536 bits
RSA-4096	4096 bits	1233 digits	~2048 bits

RSA-1024 is historical and inadequate for modern deployment. p and q are normally of comparable size.

Parameter size ≠ security strength. Approximate NIST classical mapping:

RSA-2048 → 112-bit

RSA-3072 → 128-bit

RSA-3072 and AES-128 target ≈ 128-bit classical security strength

This compares security strength — not key size, algorithm structure, or performance.

A widely used public exponent: $e = 65537 = 2^{16} + 1$ — *sparse binary form speeds up encryption.*
Typically: **Public-key operation < Private-key operation** in cost.
Private-key implementations commonly use the CRT to accelerate computation modulo p and q.

Key idea: RSA parameter size, security strength and computational cost are related — but they are not the same quantity.

RSA — SECURITY BOUNDARY

Textbook RSA Works — But It Is Not Secure Enough

What we have constructed is textbook RSA — mathematically correct:

$$C = M^e \bmod n \qquad M = C^d \bmod n$$

Mathematical Correctness ≠ Secure Deployment

TEXTBOOK RSA

$M \rightarrow C$

$M \rightarrow C$

Same M → same C

RANDOMIZED RSA ENCRYPTION

$(M, r_1) \rightarrow C_1$

$(M, r_2) \rightarrow C_2$

$C_1 \neq C_2$ (fresh randomness)

Secure RSA encryption requires randomized padding, e.g. **RSA-OAEP.**

Real-world intuition: repeated login traffic is protected via TLS session keys — not by RSA-encrypting fields directly.

Another problem — algebraic structure: $E(M_1)E(M_2) \equiv E(M_1M_2) \pmod n$

RSA-OAEP – Encryption

RSA-PSS – Signatures

Next: Secure Use of RSA

Key idea: Key size protects against computational attacks; secure encoding protects against weaknesses of textbook RSA itself.

THINK LIKE AN ENGINEER

Where Is the RSA Trapdoor?

Everyone knows **PK = (n, e)** and that **n = pq.**

So why can't an attacker simply recover the private key?

OUR TOY RSA

p = 11 q = 13

n = 143

143 = 11×13 (easy)

ϕ(n) = 120

d = 103 (e=7)

n→p,q→ϕ(n)→d

is easy.

WHERE IS THE TRAPDOOR?

(p, q)

The secret factorization is the trapdoor information

Without the trapdoor:

n →(hard)→ p,q

With the trapdoor:

p,q→ϕ(n)→d

is efficient.

NOW SCALE IT UP

RSA-1024 (historical)

308 digits

prime ≈154 digits

RSA-2048

617 digits

prime ≈308 digits

Engineering challenge:

Recover ~308-digit factors from a 617-digit n — infeasible classically.

p,q →*(multiply, easy)*→ **n = pq** but **n** →*(factor, hard at scale)*→ **p,q**

Knowing the trapdoor information gives:

p,q → ϕ(n) =(p-1)(q-1) → d = e^{-1}(mod(ϕ(n))) → Private Operation

1024 → 2048 bits does not merely make factoring "twice as hard."

Best known classical factoring algorithms do not scale linearly with modulus size.

Key idea: Multiplying p and q to create n is easy; reversing that process for a cryptographic-scale RSA modulus is the computational barrier protecting the private key.

References and Further Reading

Original RSA Paper

R. L. Rivest, A. Shamir, L. Adleman — "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," **Communications of the ACM,** Vol. 21, No. 2, 1978, pp. 120–126.

Standards and Specifications

NIST SP 800-57 Part 1 Rev. 5 — Recommendation for Key Management, Part 1: General.
NIST SP 800-56B Rev. 2 — Pair-Wise Key-Establishment Schemes Using Integer Factorization Cryptography.
RFC 8017 (PKCS #1 v2.2) — RSA Cryptography Specifications: RSA encryption, RSA-OAEP, RSA-PSS.

Course References

Jean-Philippe Aumasson, Serious Cryptography, 2nd Ed., No Starch Press.
Christof Paar and Jan Pelzl, Understanding Cryptography, Springer.

RSA combines modular arithmetic, prime structure, and computational hardness into a practical public-key system.